

When LoRA Composition Fails: Capability Retention Without Functional Composition

Nassim Berrada
Unfolding Labs
nassim@unfoldinglabs.ai

Preprint

Abstract

Parameter-space adapter merging is often discussed as a way to combine capabilities. A stronger claim is that a weighted merge of two independently trained LoRA adapters will execute an unseen transformation formed by applying task A followed by task B. We study this distinction with a controlled benchmark of deterministic structured-record transformations. The benchmark separates atomic competence, capability retention, dedicated composite training, joint training, and dynamic composition. Across five retained configuration families, dedicated composite adapters reach approximately 98–100% IID accuracy, while additive and CAT dynamic composites remain at 0% on the analyzed composite evaluations. Atomic retention varies substantially, so dynamic failure is not reducible to complete destruction of one constituent capability. Stored-adapter analysis finds attention-only updates and no simple relationship between update cosine similarity and functional composition. The result is a boundary condition: capability coexistence does not imply functional composition. We release the benchmark, predictions, qualification reports, and analysis utilities to support further study of adapter interference and ordered execution.

1 Introduction

Low-rank adapters make it inexpensive to specialize a shared model and attractive to reuse those specializations together. A natural experiment is to train an adapter for task A, train another for task B, merge them, and test a new prompt requiring both transformations. This experiment tests more than multitask retention. It tests whether a parameter-space operation can realize a new computation that neither adapter was trained to perform alone.

The distinction matters because an additive merge has the form

$$\Delta W_{\text{merge}} = \alpha \Delta W_A + \beta \Delta W_B, \quad (1)$$

whereas sequential task execution requires an ordered computation. In general, the former is not the latter. A

merged model may retain responses to A and B prompts while failing a prompt that asks for A followed by B.

We investigate this failure mode in a small, controlled setting. Inputs are key–value records and tasks are deterministic transformations such as sorting fields, selecting values, normalizing values, removing duplicates, renaming fields, and swapping keys and values. This setting makes the target program explicit and allows evaluation beyond string formatting.

Our contributions are:

- a qualification protocol separating atomic task validity from composition success;
- a controlled comparison of dedicated composite adapters, joint controls, additive merges, and CAT merges;
- an analysis of capability retention, interference, and stored LoRA update geometry; and
- a reproducible benchmark and diagnostic toolkit for studying when adapter merging preserves capabilities without composing them.

The paper does not claim that LoRA merging is generally ineffective. It reports a systematic boundary observed for independently trained adapters, structured transformation prompts, and the merge mechanisms evaluated here.

2 Research question

An *atomic task* is a single deterministic transformation learned and evaluated on its own, such as sorting fields or selecting values. A *composed task* is a new target formed by applying two such transformations in a specified order. The order matters whenever the first transformation changes the record seen by the second. A *dynamic adapter* is assembled at inference time from independently trained constituent adapters and has not been trained on the composed target. By contrast, a *dedicated adapter* is trained directly on composed examples and provides a learnability control.

The central question is whether a parameter-space combination can produce the behavior of an ordered composite that is absent from both constituent training sets. This question is distinct from whether the combination retains useful responses to the constituent prompts. We call that latter property *atomic retention*; the loss relative to standalone performance is *interference*. *Functional composition* means solving the unseen ordered target, whereas *sequential execution* is a different mechanism that explicitly preserves the intermediate computation.

The study therefore asks: under what task relationships and adapter interactions can independently trained LoRA updates coexist without destroying constituent behavior, and when—if ever—does that coexistence amount to functional composition? Additive and CAT merges provide the static parameter-space cases examined here; dedicated and joint controls establish whether the target itself is learnable.

3 Related Work

Several strands of work motivate adapter composition, but they do not all make the same operational claim. LoRA introduced low-rank updates as a parameter-efficient alternative to full fine-tuning (Hu et al., 2022). Task arithmetic combines task vectors to transfer or retain capabilities (Ilharco et al., 2022), while model soups average fine-tuned models when their parameters are compatible (Wortsman et al., 2022). TIES-Merging addresses parameter interference through trimming, sign election, and merging (Yadav et al., 2023). Adapter-Fusion instead learns a composition module over task-specific adapters (Pfeiffer et al., 2021). These methods all combine information from multiple adaptations, but they do not all promise that a fixed weighted sum will execute an ordered program.

Recent work is closer to the present question. LoRA Soups explicitly evaluates skill composition and reports strong results for concatenating LoRA updates on practical tasks (Prabhakar et al., 2024). That work motivates testing composition rather than only multitask retention, while also leaving room for task-dependent failure modes. Other approaches use task-aware retrieval or learned routing to select and combine adapters (Adsul et al., 2026; Dhasade et al., 2026; Shah and Wagle, 2026). These mechanisms can preserve task identity at inference time. Our additive and CAT experiments do not have such a router; they ask whether the static update itself contains the required composition.

Our experiment targets the strongest of these claims. It asks whether a static merge of two adapters can perform a transformation that requires both operations in a specified order, even though neither constituent adapter saw that composite target during training. A result that preserves A prompts and B prompts can support multitask coexistence without supporting this claim. A routing method might avoid the problem by selecting an

adapter from task identity; a sequential method might avoid it by preserving the order of intermediate computation. We treat those as separate mechanisms rather than as alternative names for additive merging.

This distinction also changes what counts as a useful negative result. A zero score on an unseen composite prompt does not refute model merging as a way to combine capabilities. It does show that the particular merge operation, training setup, and task interface did not turn two task vectors into an ordered program. The controls in this paper are designed to make that narrower statement testable.

4 Methodology

The methodology is designed to make a narrow causal comparison possible. Every task uses the same deterministic record generator and structured schema, so differences can be attributed to task relation, merge operation, or adapter interaction rather than ambiguous targets. The evaluation combines controlled splits, independently trained constituent adapters, learnability controls, inference-time merges, and saved diagnostics. Figure 1 summarizes these elements; the subsections define the corresponding measurements.

4.1 Data and splits

Each task has 500 training examples, 200 validation examples, and 50 examples in each test split. The training set teaches the transformation and the validation set selects checkpoints. The IID test set uses held-out records from the same generator regime. The latent test set changes held-out properties such as field counts or value offsets while preserving the task rule. The composition-OOD split changes the composite target regime and directly tests the ordered two-operation program. These splits separate memorization of the record vocabulary from generalization of the transformation and from performance on an unseen composition.

4.2 Atomic qualification

An atomic adapter is trained and evaluated on one transformation, such as sorting fields or selecting values. We qualify each constituent before interpreting a merge. Qualification requires learning the atomic IID task, generalizing to the latent split, improving over the base model, and meeting per-seed stability checks. This prevents a failed constituent from being mistaken for composition interference. It also keeps dynamic success out of the qualification decision: failure of the merge is the observation being measured.

4.3 Adapter training

The base model is Qwen2.5-0.5B-Instruct. Each independent LoRA adapter uses rank 8, alpha 16, dropout 0.05, and targets the query, key, value, and output projections in the attention blocks. Training uses two epochs, AdamW, learning rate 2×10^{-4} , batch size 8, gradient accumulation of 2, and maximum sequence length 512. For every ordered pair, independent constituent adapters are trained separately. Dedicated composite and jointly trained controls use composite examples and the same model family and training schedule. These controls distinguish a failure of merging from a failure of the target, data, or prompt interface.

4.4 Dynamic composition

A dynamic adapter is assembled at inference time from independently trained adapters and receives no composite-task training. Additive composition forms a weighted sum of the effective LoRA updates, with weights (0.25, 0.75), (0.5, 0.5), and (0.75, 0.25) in the full sweep. CAT composition concatenates the low-rank factors so that the constituent updates occupy separate rank subspaces before the resulting update is applied. Both methods are static parameter-space operations: neither explicitly executes the first task, stores its intermediate record, or routes tokens to a task-specific adapter. Sequential and routed mechanisms therefore represent different hypotheses and are not treated as interchangeable with additive or CAT composition.

4.5 Controls

The base model establishes the prompt difficulty. Each constituent adapter is evaluated on the composite prompt to measure whether one task accidentally solves part of the target. The dedicated composite adapter is trained directly on the target, while the joint control uses composite data under the standard training procedure. Finally, the dynamic merge is evaluated on both composite prompts and atomic prompts. The atomic prompts measure retention; the composite prompts measure functional composition. A high dedicated score with a low dynamic score is the most direct evidence that the target is learnable but not produced by the merge.

4.6 Evaluation

The evaluator parses each prediction into structured fields and reports exact and structured metrics. When field order is not semantically required, it compares fields as multisets, so harmless reordering does not count as an error. It separately records parse failures, empty outputs, field counts, duplicate behavior, and operation-specific outcomes. The primary composite score is exact correctness of the required structured content on the unseen composite prompt. Atomic retention is reported on

the two constituent prompts, and latent and composition-OOD results expose failures that an IID score can hide. Predictions and metadata are saved so that evaluation can be recomputed offline.

4.7 Diagnostics and analysis

The diagnostics layer checks that loading and applying an adapter does not mutate the stored weights, that repeated inference is deterministic, and that endpoint tests recover the expected behavior of each constituent adapter. It also joins predictions with parse and operation diagnostics. Stored safetensor weights are analyzed through effective updates BA , with global and module-local norms, cosine similarity, sign agreement, and high-magnitude overlap. The current adapters target attention projections only, so the module analysis records the absence of MLP and embedding updates rather than treating those groups as observed evidence.

4.8 Qualification rule

A reported configuration must pass the atomic learning and latent-generalization checks, the jointly trained composite control must solve the composite IID and latent splits, and adapter integrity, determinism, and endpoint checks must pass for every qualification seed. Dynamic success is not part of this rule. The purpose is to compare dynamic outcomes among task pairs for which the constituent tasks and the composite target are demonstrably learnable.

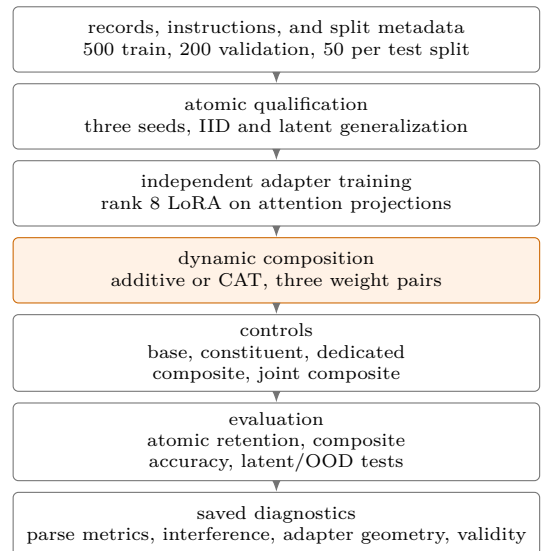


Figure 1: The evaluation pipeline used in the study. Atomic qualification comes before composition, and the dedicated and joint controls separate target learnability from the behavior of an inference-time merge. The final analysis joins predictions with validity, retention, interference, and stored-adapter diagnostics.

Table 1: Split-wise mean performance across the five analyzed pair families, both dynamic methods, all weights, and all seeds. Dynamic and semantic scores coincide after multiset-aware structured evaluation. The selection-sorting contribution uses the controlled target generator.

Split	Dynamic	Semantic	Dedicated
IID	0.0%	0.0%	100.0%
Latent	0.0%	0.0%	99.2%
Composition-OOD	0.0%	0.0%	47.1%

Table 2: Mean atomic retention and interference on the IID split. Retention is composed accuracy divided by standalone accuracy. Interference is standalone accuracy minus composed accuracy. Values average both dynamic methods, all weights, and all seeds.

Pair family	Ret. A	Ret. B	Int. A	Int. B
Renaming-swap	89.7%	77.7%	10.3%	22.3%
Sorting-deduplication	75.8%	80.0%	24.2%	20.0%
Sorting-normalization	73.7%	98.6%	26.3%	1.4%
Sorting-selection	80.6%	49.7%	19.4%	50.3%
Selection-sorting	49.7%	80.9%	50.3%	19.1%

5 Results

The study reports five retained experiment families: field renaming with key-value swap; field sorting with duplicate removal; field sorting with value normalization; field sorting with value selection; and the reverse value-selection/field-sorting order. The generated analysis contains 900 composition result rows across methods, weights, seeds, and splits.

Result: Dedicated composite adapters learn the targets, while additive and CAT dynamic adapters do not.

Evidence: Dedicated adapters learn the composite task, but independently trained adapters do not dynamically realize it. Across the five analyzed pair families, additive and CAT methods score 0% on the corrected composite evaluations, while dedicated and joint controls remain near-perfect where reported. The selection-sorting configuration also scores 0% when its target requires both selection and a substantive reordering step.

Result: Dynamic failure is often asymmetric rather than a uniform loss of both skills.

Evidence: On the sorting-selection pair, the dynamic model retains approximately 81% of sorting performance but only 50% of value-selection performance on the IID split. Reversing the order changes the pattern: selection retention is approximately 50%, while sorting retention is approximately 81%. The composite score remains 0% in both directions. The merged adapter can therefore favor one constituent behavior while failing to execute the ordered target. Composite accuracy alone would hide this

distinction; the atomic-prompt controls expose which capability is being retained and which is being overwritten or suppressed.

The analyzed set also contains both orders for value selection and field sorting. Reversing the order changes which atomic capability suffers most, but it does not produce a positive dynamic result when the target requires both operations to change the record. This is a small test of order sensitivity, not a proof that every noncommutative pair behaves the same way. It does show why reporting one direction alone would hide an important part of the failure.

Result: Atomic retention and functional composition are separate outcomes.

Evidence: Atomic retention is asymmetric and pair-dependent. In sorting-selection, value-selection retention is approximately 49–51%, while sorting retention is approximately 80%. In the reverse order, the weaker component changes position but dynamic composition remains near zero. In renaming-swap, CAT retains approximately 91% and 86% on the two atomic prompts, yet composite accuracy remains 0%. Thus, complete constituent destruction is not necessary for functional composition to fail.

These results separate two questions that are often conflated: whether a merged model still contains useful behavior, and whether it performs a new ordered program. The first can hold while the second fails.

The retained atomic behavior also gives a more useful failure description than a binary "merge failed" label. For sorting followed by selection, selection loses roughly half of its IID performance under the dynamic composite while sorting remains stronger. For the reverse order, the asymmetry flips with the adapter roles, yet the composite prompt remains unsolved. The merge can therefore contain recognizable fragments of the constituent skills without producing the intermediate representation required by the target program.

Result: Stored-update geometry does not provide a simple composition predictor.

Evidence: We extract LoRA tensors from the stored adapters and compare effective constituent updates on common modules. The current training configuration targets attention projections; no MLP adapter updates are present. Mean global/module cosine is approximately 0.16 for renaming-swap and sorting-selection, 0.23 for sorting-deduplication, and 0.39 for sorting-normalization. The highest-overlap retained pair nevertheless has 0% dynamic accuracy.

Module-level cosine ranges from negative to strongly positive values within each pair. Sign agreement and high-magnitude overlap likewise vary by module. With

only five retained pair families, these features are descriptive rather than predictive, but they rule out a simple hypothesis that higher update cosine is sufficient for functional composition.

The geometry should also be read with the target-module configuration in mind. The adapters update attention projections only, so the current measurements describe overlap in that subspace. They do not tell us whether an MLP update would preserve, overwrite, or route task-specific features. The absence of MLP updates is a limitation of the present experiment rather than evidence that MLP geometry is irrelevant.

6 Discussion

The central interpretation is not that LoRA merging is impossible. Rather, the experiment tests a stronger operation than many merging demonstrations require. Task-vector and model-merging results often target capability coexistence, transfer, or routing. A weighted sum can be useful for those goals without representing an ordered computation. Explicit routing or chaining may be required when task identity or intermediate representations must be preserved.

The negative result is therefore scientifically useful as a boundary condition. It shows that a dedicated composite control can solve the target while the parameter merge cannot, and that atomic retention is not an adequate proxy for compositionality. It also suggests that a useful predictor may need to describe where updates act and how they alter intermediate representations, rather than rely on a global vector angle alone.

7 Limitations

The study has a narrow scope. It uses Qwen2.5-0.5B-Instruct, one LoRA rank and scaling configuration, one family of structured-record tasks, and two primary parameter-space merge methods. The task vocabulary is synthetic and deliberately regular. That makes the target program and the validity checks clear, but it does not establish how the same behavior would appear in code, vision, natural-language reasoning, or larger models.

The number of retained pair families is also small. The five families cover useful differences in filtering, ordering, normalization, deduplication, renaming, and key/value transformation, yet they are not independent observations. Several share the same atomic adapters, and reverse-order experiments share the same pair geometry. Pair, seed, weight, and method are nested factors, so the current tables should not be used as if they were hundreds of independent samples.

The geometry features are limited by the saved adapter configuration. All current adapters target attention projections, and the global comparison flattens effective updates across common modules. That representation can

hide layer-specific behavior, sign cancellation in the forward pass, and interactions with the frozen base model. The reported cosine values are therefore diagnostics, not mechanistic explanations.

Finally, explicit sequential composition remains underrepresented. A sequential adapter mechanism is not equivalent to generating an intermediate textual output and feeding it into a second model, and a failure of one implementation would not settle the question. The present results are strongest for additive and CAT merging.

8 Future work

Further work should change one factor at a time. A useful geometry extension would train matched adapters targeting attention and MLP projections, then compare layer-local conflict with atomic retention. A useful task extension would add a positive control whose operation is known to be compatible with a static merge, rather than adding more arbitrary transformations. A useful evaluation extension would compare a task router, explicit chaining, and parameter merging on the same examples. These experiments would make a stronger follow-up than another broad sweep over weights.

The retained corpus supports further study of the distinction between capability coexistence and functional composition. It also provides a compact evaluation dataset and diagnostic harness for comparing parameter-space merging with explicit execution and routing.

9 Conclusion

Across five retained structured-transformation pair families, additive and CAT LoRA merges preserved some constituent behavior in some settings but failed to implement the unseen ordered composite. Dedicated composite adapters solved the same targets, and update geometry did not yield a simple explanation based on cosine similarity. The appropriate conclusion is a distinction: capability coexistence is not functional composition.

The resulting benchmark provides task configurations, predictions, qualification reports, adapter tensors, and analysis scripts. These materials support direct comparison of parameter-space merging with explicit ordered execution and routing.

A Appendix

A.1 Qualification thresholds

Atomic qualification required mean IID accuracy of at least 0.75, mean latent accuracy of at least 0.60, an IID gain over the frozen base model of at least 0.15, IID and latent seed standard deviations no greater than 0.10, per-seed IID accuracy of at least 0.65, and per-seed latent accuracy of at least 0.50. Composite qualification required

constituent accuracy on the composite prompt to remain at or below 0.40 on both IID and latent splits, joint and dedicated composite accuracy of at least 0.75 on IID data, and dedicated latent accuracy of at least 0.50. Adapter endpoint checks used an accuracy-difference tolerance of 0.001. Determinism and adapter-mutation checks were required to pass.

A.2 Diagnostic analysis

The stored diagnostics were grouped into four levels. Prediction diagnostics record parseability, empty outputs, predicted and target field counts, exact and multiset-aware content matches, duplicate predictions, content precision, recall, and F1. Operation diagnostics compare each prediction with the source record, the intermediate record, and the final target, both in order and as unordered pairs. Composition diagnostics summarize base, constituent, dynamic, dedicated, and joint accuracy together with composition gain and deficit. Adapter diagnostics inspect the saved LoRA tensors, form effective updates BA , and report tensor norms, parameter counts, module counts, global and module-local cosine similarity, sign agreement, high-magnitude overlap, and relative update norms. These measurements are joined by pair, method, seed, split, and weight so that output failures can be separated from retention loss and stored-update geometry.

A.3 Task inventory

The benchmark uses one input schema, a record containing ordered key-value fields. The atomic task registry contains the following transformations.

A.4 Atomic tasks

Every atomic adapter receives the same record schema, but each task changes one aspect of the record or its representation.

- **Structured extraction.** Copies every key-value field. For example, `a: echo, b: hotel` remains unchanged.
- **Structured classification.** Inspects the record and returns a binary class such as YES or NO according to the task rule.
- **Field filtering.** Keeps fields at selected positions, such as the odd-positioned fields `a` and `c`.
- **Value selection.** Keeps fields whose values begin with a vowel, such as retaining `a: echo` while dropping `b: hotel`.
- **Field sorting.** Reorders fields by ascending value, so `b: alpha, a: echo` becomes `b: alpha, a: echo` regardless of input order.

- **Field reversal.** Reverses the input field order.
- **Duplicate removal.** Keeps the first occurrence of each value when multiple fields share a value.
- **Value normalization.** Lowercases each value, for example changing `Echo` to `echo`.
- **Field renaming.** Replaces keys with positional names such as `field_1`, `field_2`, and so on.
- **Key-value swap.** Exchanges the key and value in every pair, turning `a: echo` into `echo: a`.
- **Key prefixing.** Prepends `task_` to each key, turning `a` into `task_a`.
- **Key suffixing.** Appends `_task` to each key, turning `a` into `a_task`.
- **Value prefixing.** Prepends `value_` to each value, turning `echo` into `value_echo`.
- **Value suffixing.** Appends `_value` to each value, turning `echo` into `echo_value`.

A.5 Atomic instruction prompts

The following are the complete task instructions used by the atomic examples. In each prompt, `<record>` is replaced by the generated record, `<format>` is replaced by the selected output format, and `<answer>` marks the beginning of the expected answer. The optional noise prefix used by some splits is prepended to this instruction and does not change the task rule.

- **Structured extraction.**

Extract every key-value field from the record. Return all fields and do not omit any field. Record: `<record>`. Format the answer as `<format>`. For "lines", output one "key: value" pair per line. For "comma", output "key=value" pairs separated by commas. Answer:

- **Structured classification.**

Classify the record as YES if at least one field value is exactly "alpha"; otherwise classify it as NO. Record: `<record>`. Return exactly one label: YES or NO. Answer:

- **Field filtering.**

Filter the record by keeping only fields in odd-numbered positions: 1st, 3rd, 5th, and so on. Discard all even-numbered positions. Record: `<record>`. Format the answer as `<format>`. Answer:

- **Value selection.**

Select every field whose value begins with a vowel (A, E, I, O, or U). Discard all other fields. Record: <record>. Return the selected fields as comma-separated key=value pairs. If no field qualifies, return NONE. Answer:

- **Field sorting.**

Sort all fields alphabetically by their values in ascending order. Record: <record>. Format the answer as <format>. Preserve each key-value pair exactly. Answer:

- **Field reversal.**

Reverse the sequence of fields in the record from right to left, so the last field comes first and the first field comes last. Record: <record>. Format the answer as <format>. Answer:

- **Duplicate removal.**

Remove duplicate values from the record. Keep the first occurrence of each value and remove later fields having a value that has already appeared. Record: <record>. Return the remaining fields as comma-separated key=value pairs. Answer:

- **Value normalization.**

Normalize every field value by converting it to lowercase. Preserve all keys and all fields. Record: <record>. Format the answer as <format>. Answer:

- **Field renaming.**

Rename every field key using this mapping: the 1st field becomes field_1, the 2nd field becomes field_2, the 3rd field becomes field_3, and so on. Preserve every value exactly. Record: <record>. Format the answer as <format>. Answer:

- **Key-value swap.**

Swap the key and value of each field in the record. Each original value becomes the key, and each original key becomes the value. Record: <record>. Format the answer as <format>. Answer:

- **Key prefixing.**

Add the fixed marker "task_" before every field key. Record: <record>. Format the answer as <format>. Answer:

- **Key suffixing.**

Add the fixed marker "_task" after every field key. Record: <record>. Format the answer as <format>. Answer:

- **Value prefixing.**

Add the fixed marker "value_" before every field value. Record: <record>. Format the answer as <format>. Answer:

- **Value suffixing.**

Add the fixed marker "_value" after every field value. Record: <record>. Format the answer as <format>. Answer:

A.6 Composed tasks

The composed generator applies two of these transformations in order and renders the resulting record in the same line or comma formats. The reported families are:

- **Renaming–swap.** Renames the keys and then exchanges keys and values. An input `a: echo` first becomes `field_1: echo`, after which the pair is swapped.
- **Sorting–deduplication.** Sorts fields by value and then removes repeated values. The second operation sees the sorted intermediate record.
- **Sorting–normalization.** Sorts by value and then lowercases the values.
- **Sorting–selection.** Sorts by value and then keeps only vowel-initial values.
- **Selection–sorting.** Keeps vowel-initial values and then sorts the retained fields by value. This reverses the previous order and tests whether the intermediate representation changes the result.

References

- Hu et al. *LoRA: Low-Rank Adaptation of Large Language Models*. ICLR, 2022. arXiv:2106.09685.
- Pfeiffer et al. *AdapterFusion: Non-Destructive Task Composition for Transfer Learning*. EACL, 2021. arXiv:2005.00247.
- Wortsman et al. *Model Soups: Averaging Weights of Multiple Fine-Tuned Models Improves Accuracy Without Increasing Inference Time*. ICML, 2022. arXiv:2203.05482.

- Ilharco et al. *Editing Models with Task Arithmetic*. arXiv:2212.04089, 2022.
- Yadav et al. *TIES-Merging: Resolving Interference When Merging Models*. NeurIPS, 2023. arXiv:2306.01708.
- Prabhakar et al. *LoRA Soups: Merging LoRAs for Practical Skill Composition Tasks*. arXiv:2410.13025, 2024.
- Adsul et al. *Task-Aware LoRA Adapter Composition via Similarity Retrieval in Vector Databases*. arXiv:2602.21222, 2026.
- Dhasade et al. *Effective LoRA Adapter Routing using Task Representations*. arXiv:2601.21795, 2026.
- Shah and Wagle. *MoLoRA: Composable Specialization via Per-Token Adapter Routing*. arXiv:2603.15965, 2026.
- Yadav et al. *Model Merging in the Era of Large Language Models*. arXiv:2403.13257, 2024.